

Looking Both Ways

Lecture Notes — Week 7: Doubly & Circular Lists, List-Backed Stack and Queue, and Polynomials

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 3, 2026

7.1 Why This Week Exists: What a Backward Link Buys

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 3, pp.74–162 (PDF) — doubly and circular lists; Wirth, *Algorithms and Data Structures*, Sec. 4.3, p. 115 — the linear-list links and the invariant that keeps them agreeing.

Four questions organize this chapter, stated here exactly as the deck states them: what does a backward link (`p->prev`) buy that one link cannot; when a chain has no `NULL`, where does a walk stop; can the *same* stack and queue contracts run on chains instead of arrays; and why does $5x^{100} + 3x^2 + 1$ want three nodes rather than 101 array slots. Compressed to a plan: price the backward link, meet the ring, re-implement the stack and queue on chains, then spend linked lists on the syllabus’s polynomial.

The handoff from Week 6 is exact. Week 6 built the singly linked list: a node is data plus one link, `head` is the sole entry, and `NULL` terminates the chain. Front-insert is $O(1)$ worst case; end-insert, delete, and search are $O(n)$ worst case; the three-pointer reverse runs in $O(n)$ time and $O(1)$ space. It closed on one promise — “what does a backward link buy?” — and named Week 7 as doubly and circular lists, the list-based stack and queue, and polynomials. The deficiency it left behind is the whole motivation: *every* delete paid the walk, because to unlink a node we first had to find its predecessor. New symbols today are the predecessor link `p->prev`, the queue’s last-node handle `tail`, and the circular-list conventions (`last->next == head`; no `NULL`).

The calculator pipeline makes the need concrete again. The expression processor’s symbol table is a linked list of identifiers (Week 6). When a block scope exits, the compiler must delete every identifier declared in it — and it already holds a pointer to each one, not to its predecessor. Singly, “delete by pointer” still costs a walk to find the node whose `next` points at the target: $O(n)$ worst case, re-finding information we already had. The same need recurs everywhere: text-editor undo/redo, browser back and forward, OS free-lists of memory blocks, and LRU caches. Two arc notes close the framing. Week 5’s circular queue advanced an index by $(\text{rear} + 1) \bmod n$; today the same wraparound appears as a linked ring. And Week 10 will replay this very symbol table as a hash table, $O(n)$ as a list against $O(1)$ expected hashed. The course design principle fires again: every structure answers a deficiency the previous one just exhibited — Week 6’s singly list had no backward walk, and the doubly linked list is the fix.

7.2 Doubly Linked Lists: The Second Link and Its Price

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — the doubly-linked node and its insert/delete pointer surgery; Wirth, Sec. 4.3, p. 115 — the two-link invariant a correct operation must restore.

7.2.1 The node that looks both ways

A user clicks *delete* on a bookmark, and the program holds a pointer straight to that node. Singly, the only way to unlink it is to find the node whose **next** points at it — a full walk, $O(n)$ worst case. The work is avoidable: at deletion we can already reach the node, so we should be able to reach its neighbours too. The fix costs exactly one extra pointer per node: store the predecessor as well as the successor.

Definition (Doubly linked list (Karumanchi Ch. 3, pp.74–162 PDF; Wirth Sec. 4.3, p. 115)). A *doubly linked list* (DLL) is a chain of nodes, each holding data plus two links: *next* to the successor and *prev* to the predecessor. *head* points to the first node; the first node's *prev* is *NULL* and the last node's *next* is *NULL*. [3, 5]

In C the node is:

```
struct node { int data; struct node *prev; struct node *next; };
```

Figure 7.1 draws the anatomy. A boxed **prev**-field sits to the left and a boxed **next**-field to the right of the **data** field (drawn holding 42); the whole node is flanked by its predecessor box on the left and its successor box on the right, joined by two-headed arrows. A blue **p** arrow points at the data field, and the labels **p->prev** and **p->next** name the two moves the node now offers. One node therefore offers *two* constant-time moves — forward through **p->next** and backward through **p->prev** — and that is all the second link does, and all the delete fix needs.

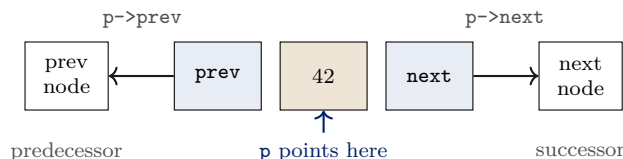


Figure 7.1: A doubly-linked node: data flanked by **prev** and **next** fields, reachable forward and backward in $O(1)$ from a pointer **p** (cf. Karumanchi, Ch. 3, pp.74–162 PDF).

Example 7.2.1 (Worked example: building $10 \leftrightarrow 20 \leftrightarrow 30$). Start empty (*head* = *NULL*). Build the chain term by term, each term one *malloc* plus link repairs whose count is the price of the second pointer. Insert 10 at the front: the new box gets *prev* = *NULL*, *next* = *NULL*, and *head* swings onto it — two link writes. Insert 20 at the front: the new box's *next* points at the old first node (10) and its *prev* to *NULL*; then the old first node's *prev* must be repointed from *NULL* to the new box; then *head* swings over — three link writes, because an interior neighbour had to be repaired. Insert 30 at the end: walk to the old last node (20), set the fresh 30-box's *prev* to it and *next* to *NULL*, then set the old last node's *next* to the fresh box — again an interior-neighbour repair. Final heap: *head* $\leftrightarrow$ [10] $\leftrightarrow$ [20] $\leftrightarrow$ [30], where each $\leftrightarrow$ is two links. The bill is exact: one extra pointer per node, and every insert or delete must repair two links instead of one.

7.2.2 Insert at the front, end, and after a given node

Pointer surgery for inserting x after a given node p (Karumanchi Ch. 3, pp.74–162 PDF):

```
newNode = malloc(sizeof(struct node)); newNode->data = x
newNode->next = p->next          // 1. wire forward
newNode->prev = p                // 2. wire backward
if (p->next != NULL) p->next->prev = newNode // 3. fix old successor
p->next = newNode               // 4. splice in
```

Each step is load-bearing. Step 1 gives the new node its successor — the old $p \rightarrow \text{next}$ — while that value is still readable. Step 2 gives it its predecessor, p itself. Step 3 repairs the old successor’s backward link so that it now points at the new node; it is guarded by $p \rightarrow \text{next} \neq \text{NULL}$ because if p was the last node there is no successor to repair. Step 4 finally overwrites $p \rightarrow \text{next}$: order matters here exactly as it did in Week 6’s front-insert, because step 3 still needs the old successor value that step 4 destroys. Front-insert and end-insert are the special cases: front-insert sets $\text{newNode} \rightarrow \text{prev} = \text{NULL}$ and moves head ; end-insert is just “insert after the old last node.”

Definition (The two-link invariant (Wirth Sec. 4.3, p. 115; standard treatment)). *After any operation, every interior node p satisfies $p \rightarrow \text{next} \rightarrow \text{prev} == p$ and $p \rightarrow \text{prev} \rightarrow \text{next} == p$. Every operation must restore this before it returns.*

Steps 3 and 4 above are precisely what keep the invariant. If you wire $p \rightarrow \text{next} = \text{newNode}$ but forget $p \rightarrow \text{next} \rightarrow \text{prev} = \text{newNode}$, then the old successor still points *backward* at p , and the two links silently disagree; a later backward walk either skips the new node or corrupts the chain. The invariant is the single condition that certifies a DLL operation is correct.

Figure 7.2 shows the final state after inserting a gold new node X (holding 20) between A (10) and B (30). Reading the four pointer writes off the picture: $X \rightarrow \text{next} = B$, $X \rightarrow \text{prev} = A$, then $B \rightarrow \text{prev} = X$ and $A \rightarrow \text{next} = X$. The old $A \leftrightarrow B$ link has been overwritten by the pair of two-headed arrows through X ; the lesson the diagram encodes is to wire the new node *before* repointing its neighbours — switch the order and either A ’s or B ’s neighbour address is lost.

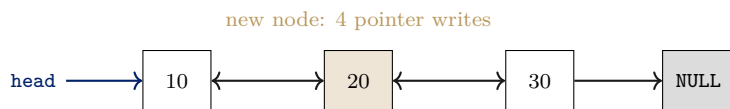


Figure 7.2: Inserting X (20) after A (10): four pointer writes restore both links of the two-link invariant (cf. Karumanchi, Ch. 3, pp.74–162 PDF).

Example 7.2.2 (Worked trace: insert 20 after 10 in $\text{head} \leftrightarrow 10 \leftrightarrow 30$). *malloc the new box and fill data = 20. Step 1: $X \rightarrow \text{next} = p \rightarrow \text{next}$, where p is the 10-node — so X now points at 30. Step 2: $X \rightarrow \text{prev} = p$ — X points backward at 10. Step 3: $p \rightarrow \text{next}$ is 30 (not NULL), so set $30 \rightarrow \text{prev} = X$ — the old successor’s backward link now names X . Step 4: $10 \rightarrow \text{next} = X$ — splice complete. Final heap reads $\text{head} \leftrightarrow 10 \leftrightarrow 20 \leftrightarrow 30$, and both halves of the invariant hold: $20 \rightarrow \text{next} \rightarrow \text{prev} = 30 \rightarrow \text{prev} = 20$, and $20 \rightarrow \text{prev} \rightarrow \text{next} = 10 \rightarrow \text{next} = 20$. Now the failure mode: swap steps 3 and 4 and step 3 sets $X \rightarrow \text{prev}$ on the wrong node (or reads $p \rightarrow \text{next} = X$, a self-loop), leaving 30’s backward link pointing at 10 forever — a backward walk from 30 reaches 10 and skips 20 entirely. Four writes, and the order is the whole operation.*

7.2.3 Delete by pointer: the $O(1)$ bypass

Deleting the node p points at (Karumanchi Ch. 3, pp.74–162 PDF):

```

if (p->prev != NULL) p->prev->next = p->next
else head = p->next           // p was the first node
if (p->next != NULL) p->next->prev = p->prev
free(p)                       // reclaim exactly once

```

Socratic pause, exactly as the deck poses it: with only `p` in hand, which two nodes must learn the new link — and how does each find its neighbour without a walk? The answer is the whole payoff: they are `p->prev` and `p->next`, both reachable in $O(1)$ straight from `p`. The first line stitches `p->prev` to `p->next` going forward (guarded because if `p` is the first node there is no predecessor, and instead `head` must move forward). The second line stitches `p->next` back to `p->prev` (guarded because if `p` is the last node there is no successor to repair). Then `free(p)` reclaims exactly once. So delete is $O(1)$ worst case once `p` is known — against Week 6’s $O(n)$.

Figure 7.3 draws deleting the middle node `D` (holding 20) from `A` (10) $\leftrightarrow$ `D` $\leftrightarrow$ `B` (30). The two dashed red links (`A` $\leftrightarrow$ `D` and `D` $\leftrightarrow$ `B`) are the ones that die; one green bypass replaces them, drawn as a curve under the row that jumps from `A` to `B`. In pointer terms the bypass is the single line `p->prev->next = p->next; p->next->prev = p->prev`, after which `free(p)` returns `D` to the heap.

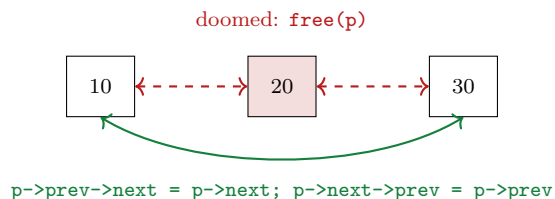


Figure 7.3: Deleting the middle node 20: the two dashed $\leftrightarrow$ links die and one green bypass joins 10 and 30 in both directions (cf. Karumanchi, Ch. 3, pp.74–162 PDF).

Example 7.2.3 (Worked trace: delete 15 from `head` $\leftrightarrow$ 5 $\leftrightarrow$ 15 $\leftrightarrow$ 25). *Given only a pointer `p` at the 15-node. First line: `p->prev` is the 5-node (not `NULL`), so set `5->next = p->next` = the 25-node. Second line: `p->next` is the 25-node (not `NULL`), so set `25->prev = p->prev` = the 5-node. Third line: `free(p)` returns the 15-node to the heap. Final heap: `head` $\leftrightarrow$ 5 $\leftrightarrow$ 25, and the invariant holds (`5->next->prev = 25->prev = 5`). The two links rewritten were 5’s `next` and 25’s `prev`, both reached in $O(1)$ from `p` — no walk, $O(1)$ worst case. Touch `p` after the `free` and it is Week 1’s dangling pointer; skip the `free` and the 15-node leaks, invisible because nothing points at it.*

7.2.4 The full price list

The invariant is the pricing engine: it explains why some operations got cheaper and some did not. The complete Week 7 bill, every bound naming its operation and its case:

operation	bound	note
insert at front	$O(1)$ worst case	2 writes; order matters
insert after given <code>p</code>	$O(1)$ worst case	given pointer skips the walk
delete given <code>p</code>	$O(1)$ worst case	the backward link’s payoff
insert/delete by value	$O(n)$ worst case	the search dominates
search	$O(n)$ worst / $O(1)$ best	unordered; <code>prev</code> does not help
display forward / backward	$\Theta(n)$	must visit all n
reverse	$O(n)$ worst, $O(1)$ space	one flip + one <code>prev</code> fix per node

Search is unchanged: $O(n)$ worst case and $O(1)$ best case — the backward link does not help an unordered search, because backward and forward walks cost the same per hop. `displayBwd` needs a `tail` handle to start from the end; reverse is Week 6’s three-pointer flip plus one `prev` fix per node. The lesson is the shape of the table: only the operations handed a node pointer outright become $O(1)$; anything that must *find* a node still pays the linear search, and the second link merely doubles the per-node storage and the per-operation link maintenance.

7.3 Circular Linked Lists: Rings With No End

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — circular single and double lists and their traversal termination.

7.3.1 A ring with no end

A round-robin scheduler hands the CPU to each process in turn and then returns to the first; a repeating playlist does the same; so does a token-ring network. The last element is followed by the first — there is no natural end. A chain that models this should make the last node point back to the first instead of to `NULL`.

Definition (Circular linked list (Karumanchi Ch. 3, pp.74–162 PDF)). A *circular linked list* is a chain whose last node’s *next* points back to *head* instead of *NULL*. In a *singly circular list* each node has one link; in a *doubly circular list* *prev* and *next* both close the ring (*head*→*prev* is the last node). An empty circular list has *head* == *NULL*; a one-node circular list points to itself. [3]

Figure 7.4 draws the two rings side by side. On the left, three nodes *a*, *b*, *c* sit on a circle joined by one-way arcs, so a walk moves *a* → *b* → *c* → *a* forever; the annotation records the rule that “last next = head; no `NULL`.” On the right, the same three nodes are joined by two-headed arcs, so a walk moves both ways around the ring; the annotation records that *prev* and *next* both close the ring. A blue *head* arrow enters each ring at node *a*. The picture’s point is that a ring has no terminus to draw — so the walk must name a node to stop on.

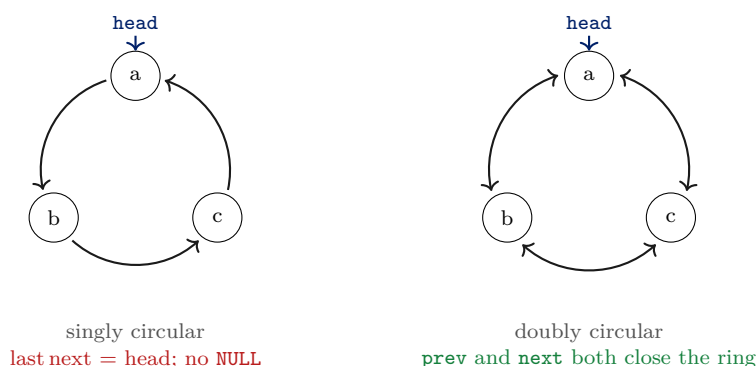


Figure 7.4: Two rings: a singly circular list (left, one-way arcs, last link home) and a doubly circular list (right, two-way arcs) (cf. Karumanchi, Ch. 3, pp.74–162 PDF).

7.3.2 Traversal without `NULL`

The termination rule (Karumanchi Ch. 3, pp.74–162 PDF):

```

p = head
if (p != NULL) do { visit p; p = p->next } while (p != head)
// stop when the walk RETURNS TO HEAD, not when it hits NULL

```

The subtlety is that a `while (p != NULL)` loop on a circular list never meets `NULL`: it loops forever or crashes. The stop condition must name `head`, and it must be checked *after* the first visit (a `do--while`) so the head node is not skipped — the loop body runs once, then the test compares against `head`. For the doubly circular ring the same rule with `p->prev` walks the ring the other way. A consequence is that every operation ending at the “last” node must handle wraparound: insert and delete must keep the ring closed by repairing both `head->prev` and `last->next`.

Rings are not a curiosity — they are exactly what round-robin schedulers, repeating playlists, and token-ring networks need, and the circular queue’s modular index is the same idea seen as a linked ring (Week 5’s $(\text{rear} + 1) \bmod n$ become one link step around the ring).

Example 7.3.1 (Check your prediction: `while (p != NULL)` on a 4-node ring). *A singly circular list holds four nodes: `head` $\rightarrow a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$. You run `while (p != NULL) { p = p->next; }`. Options: (A) visits `a, b, c, d` then stops; (B) loops forever; (C) stops after one node. The deck’s answer is **B**. Option A would hold only if some `next` were `NULL`, but in a ring none is; option C misreads the loop — the condition is checked every hop, not once. Because no node ever holds `NULL`, the test never fires and the walk cycles `a, b, c, d, a, b, c, d, ...` indefinitely. The fix is to test `p != head` (or to visit with a `do--while` so the first node is not skipped).*

7.4 Stack and Queue, Reborn on Chains

Read alongside: Karumanchi, Ch. 4, pp.163–204 (PDF) — the linked stack; Ch. 5, pp.205–223 (PDF) — the linked queue; Sedgewick & Wayne, *Algorithms*, Sec. 1.3, p. 120 — stack and queue as collection ADTs.

One ADT, implemented twice: the same Week 3 stack contract, now on chains. The point of this section is that the contract does not change — only the implementation does, and therefore only the cost profile moves.

7.4.1 List-based stack: push and pop at the head

A stack over a linked list (Karumanchi Ch. 4, pp.163–204 PDF; Sedgewick Sec. 1.3, p. 120):

```

push(x): newNode = malloc(...); newNode->data = x
        newNode->next = head; head = newNode      // O(1) worst
pop():   if (head == NULL) error                    // stack underflow
        temp = head; head = head->next
        x = temp->data; free(temp); return x       // O(1) worst

```

The top of the stack is `head`. Reading the code against Week 6: `push` is exactly Week 6’s front-insert, and `pop` is exactly Week 6’s front-delete — both rewrite one link (plus `head`), so both are $O(1)$ worst case. The underflow check is the contract’s edge case: `pop` on an empty stack is an error, not a `NULL`-dereference. The array version (Week 3) was also $O(1)$ per operation; the list’s win is unbounded size and no amortized reallocation, at the cost of $O(n)$ space for n items (one node each) and the pointer-chasing locality that comes with it. [3, 4]

Figure 7.5 draws a list-based stack. `head` (blue) enters the gold-filled node 30 — the top of the stack, since `top = head` — which links down to 20, then 10, then `NULL`. The annotation records that `push/pop` touch only `head`, at $O(1)$ worst case. After `push(40)`, `head` points at the new 40-node

and its **next** is the old top (30); **pop** reverses exactly that, returning the old top and freeing its box.

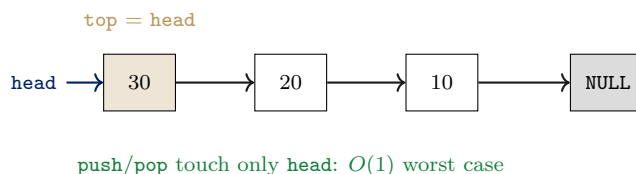


Figure 7.5: A list-based stack: **head** is the top; **push** front-inserts and **pop** front-deletes, both $O(1)$ worst case (cf. Karumanchi, Ch. 4, pp.163–204 PDF).

Example 7.4.1 (Worked trace: **push**(40) then **pop**() on $\text{head} \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$). ***push**(40):* *malloc* a box, set *data* = 40, set its *next* = *head* (the 30-node), then park *head* on the new box. Heap now reads $\text{head} \rightarrow 40 \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$; the top is 40, two pointer writes, $O(1)$ worst case. ***pop**():* *head* is not *NULL*, so *temp* = *head* (the 40-node), *head* = *head*->*next* (the 30-node), read *temp*->*data* = 40, *free*(*temp*), return 40. Heap is back to $\text{head} \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$ — exactly the prior state, showing **pop** is the precise inverse of **push**. Called once more on the now-empty stack (all four popped), *head* is *NULL* and **pop** raises underflow instead of dereferencing.

7.4.2 List-based queue: keep a tail pointer

A queue over a linked list (Karumanchi Ch. 5, pp.205–223 PDF; Sedgewick Sec. 1.3, p. 120):

```
enqueue(x):
    newNode = malloc(...); newNode->data = x
    newNode->next = NULL
    if (head == NULL) head = tail = newNode
    else tail->next = newNode; tail = newNode    // O(1)
dequeue(): if (head == NULL) error           // underflow
            temp = head; head = head->next
            if (head == NULL) tail = NULL
            x = temp->data; free(temp); return x
```

enqueue appends at the **tail** and advances it: first link the new node after the current last node, then move **tail** onto it. The empty-list branch sets both **head** and **tail** to the single new node. **dequeue** removes at **head** — the oldest element, the one next to leave — and has one extra guard: if removing the last item empties the queue, **tail** must be cleared to *NULL* along with **head**, or the next **enqueue** would link onto freed memory. Keeping **tail** makes end-insert $O(1)$, so both ends are $O(1)$ worst case — unlike a fixed array. [3, 4]

Figure 7.6 draws a list-based queue. **head** (blue) enters 10, the oldest item; the chain runs $10 \rightarrow 20 \rightarrow$ the gold-filled 30, with **tail** (gold arrow from above) pointing at 30, the newest item; then *NULL*. The annotations record the two ends: **dequeue** from **head**, **enqueue** after **tail**. **enqueue** links after **tail** and advances it; **dequeue** unlinks **head**.

Example 7.4.2 (Worked trace: **dequeue**() then **enqueue**(40) on $\text{head} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$, *tail* at 30). ***dequeue**():* *head* is not *NULL* (*temp* = the 10-node), then *head* = *head*->*next* (the 20-node). *head* is not *NULL*, so *tail* stays on 30. Read 10, *free* the 10-node, return 10. Heap: $\text{head} \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$, *tail* still 30. ***enqueue**(40):* *malloc* a box with *data* 40 and *next* *NULL*;

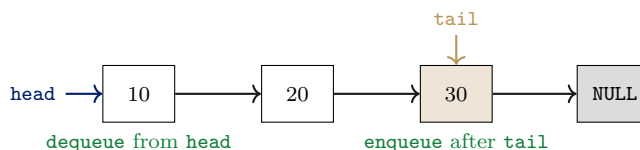


Figure 7.6: A list-based queue: **dequeue** unlinks **head** (oldest), **enqueue** links after **tail** (newest); both ends $O(1)$ worst case (cf. Karumanchi, Ch. 5, pp.205–223 PDF).

*head is not NULL, so $tail \rightarrow next$ (30's next) = 40, then $tail$ = the 40-node. Heap: $head \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow NULL$, $tail$ now 40. Now the emptying case: from $head \rightarrow 20 \rightarrow NULL$ with $tail$ on 20, **dequeue()** sets $head = NULL$ and then, because the new $head$ is $NULL$, also sets $tail = NULL$ — without that guard, the next **enqueue** would write through a stale $tail$ into freed memory.*

7.4.3 Array vs. list: the honest bill

operation	array	linked list
stack push/pop	$O(1)$ worst, fixed capacity*	$O(1)$ worst, unbounded
queue enqueue/dequeue	$O(1)$ worst (circular), fixed	$O(1)$ worst (head+tail), unbounded
memory	one block of n slots	one node per item, +1 pointer each
locality	contiguous, cache-friendly	scattered, pointer-chasing

*Stack reallocation is amortized $O(1)$ when it grows. There is no universal winner: choose the array when the size is known or bounded and locality matters; choose the list when the size is unpredictable and growth is frequent. The contract is identical — only the cost profile moves, which is exactly the abstraction the ADT was supposed to give us.

Circular-queue variant: keeping $tail \rightarrow next == head$ closes the ring, so one $tail$ pointer gives round-robin **enqueue** and **dequeue** at $O(1)$ worst case — no count, no sacrificed slot, no mod (Week 5's linked cousin). The single handle $tail$ reaches the front through $tail \rightarrow next$, so both the enqueue end and the dequeue end are one link from a pointer we already hold.

7.5 Polynomial Manipulation

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 4 (chapter-level, standard treatment for polynomial representation and addition); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 2 (chapter-level, standard treatment).

7.5.1 Why polynomials want a list

The polynomial $5x^{100} + 3x^2 + 1$ has *three* nonzero terms but spans exponents 0 to 100. An array indexed by exponent needs 101 slots — 98 of them holding zero. Sparse data, meaning mostly empty, is exactly when a contiguous array wastes space; storing only the nonzero terms as (coefficient, exponent) nodes keeps three nodes. The calculator's evaluator can hold a polynomial expression as a chain, then add, subtract, or multiply two such chains term by term. The same deficiency pattern fires once more: the array has a fixed span, while the list stores only what actually exists.

7.5.2 The polynomial node

Definition (Polynomial node (Horowitz & Sahni Ch. 4, standard treatment; Aho–Hopcroft–Ullman Ch. 2, standard treatment)). A polynomial $\sum_i a_i x^{e_i}$ is stored as a list of nodes, each holding a coefficient and an exponent, kept in *strictly descending exponent order* (the course convention). [2, 1]

In C:

```
struct pnode { float coeff; int exp; struct pnode *next; };
```

The head handle is again `head`; the zero polynomial is `head == NULL`. Terms whose coefficient is 0 are never stored — that is the sparsity payoff. For example, $5x^3 + 4x + 2$ is the chain $(5, 3) \rightarrow (4, 1) \rightarrow (2, 0) \rightarrow \text{NULL}$, and the ordering by descending exponent is what makes addition a single linear merge rather than a repeated search.

7.5.3 Addition: merge by descending exponent

Adding two ordered chains (Horowitz & Sahni Ch. 4, standard treatment):

```
// A, B sorted by descending exp; build C
while (A != NULL && B != NULL):
    if A->exp > B->exp: append copy of A; A = A->next
    elif B->exp > A->exp: append copy of B; B = B->next
    else: sum = A->coeff + B->coeff
        if (sum != 0) append node(sum, A->exp)
        A = A->next; B = B->next
append the rest of whichever list is not exhausted
```

The two ordered chains are merged like two sorted runs. One pass walks both lists, advancing whichever head has the larger exponent and copying it through; when the exponents are equal the coefficients are added and, if the sum is nonzero, one node is emitted; when one list is exhausted the remainder of the other is appended whole. The output is itself in descending exponent order, so the invariant is preserved at every step. The cost is $O(n+m)$ worst case for n and m terms in A and B: each source node is visited exactly once and produces at most one output node. Appending to the output’s tail is Week 6’s end-insert; keeping an output `tail` pointer makes each append $O(1)$, so the whole merge is linear. [2]

Figure 7.7 draws the alignment that makes the merge obvious. Three rows share four exponent columns labelled `exp 3`, `exp 2`, `exp 1`, `exp 0` (left to right). Row A holds $5x^3$, $4x$, 2 ; row B holds $3x^2$, $4x$, 1 ; row C holds $5x^3$, $3x^2$, $8x$, 3 . The gold cells mark shared exponents: the $4x$ in A and the $4x$ in B sit in the `exp 1` column, and their sum appears as the gold $8x$ in C’s `exp 1` column. The caption under the figure states the rule the gold column encodes: equal exponents add coefficients ($4 + 4 = 8$) and keep the exponent.

Example 7.5.1 (Worked example: add $A = 5x^3 + 4x + 2$ and $B = 3x^2 + 4x + 1$). Walk the merge step by step, tracking C so far:

<i>step</i>	<i>compare</i>	<i>action</i>	<i>C so far</i>
1	3 vs. 2	A’s exp larger	$5x^3$
2	1 vs. 2	B’s exp larger	$5x^3 + 3x^2$
3	1 vs. 1	equal: $4 + 4$	$5x^3 + 3x^2 + 8x$
4	0 vs. 0	equal: $2 + 1$	$5x^3 + 3x^2 + 8x + 3$

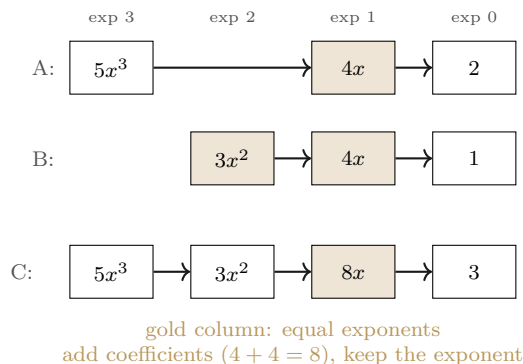


Figure 7.7: Adding $A = 5x^3 + 4x + 2$ and $B = 3x^2 + 4x + 1$ by descending exponent: shared (gold) exponents add coefficients; unshared terms copy through.

Read the compare column: at each step the larger exponent is copied through, and equal exponents are added. No term is dropped and no zero term is stored. A cancellation (say $4x + (-4x)$) would append nothing — the sparsity discipline in action.

Multiplication repeats the idea: each term of A times every term of B (coefficients multiply, exponents add), then the resulting terms are merged equal-exponent by equal-exponent — $O(n \cdot m)$ worst case for n terms of A and m terms of B . **Verdict:** the array wins on a small, dense exponent range ($O(1)$ term lookup by exponent); the list wins on a sparse, high-degree polynomial. State the workload, then choose.

7.6 Unit-II Review and Close

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — linked lists, the spine of Unit II (this review is the deck’s own, not a new citation).

Unit II in one map, built for Class Test 2:

week	contract	load-bearing skill
6	singly list: head , p->next , NULL	pointer surgery; three-pointer reverse
7	DLL p->prev ; rings; list stack/queue; polynomials	two-link invariant; ring termination; ADT reuse; merge

The decision array-versus-list is now a full comparison rather than “lists are better”: positional reads favour the array; anywhere-edits and unpredictable size favour the chain. The test rewards traces — draw the chain, move the pointers, then name the operation, the bound, and the case.

Three classic mistakes, stated exactly as the deck states them. **Forgetting half the invariant** — updating **p->next** but not **p->next->prev** (or the reverse). The two links silently disagree, and a later backward walk corrupts or skips nodes. **Waiting for NULL in a ring** — **while** (**p != NULL**) on a circular list never terminates; the stop condition must name **head**. **Ignoring the endpoints** — forgetting to move **head** (or clear **tail**) when the first or last node leaves, or reusing a pointer after **free**. This is Week 1’s dangling pointer and leak, now wearing list clothing. How to revise: re-trace insert-after and delete-by-pointer with a covered answer column, then trace one polynomial addition — the test is traces, not definitions.

In summary: **doubly linked list** — node = data + **prev** + **next**; delete and insert by a known pointer are $O(1)$ worst case, paid for with an extra pointer per node and a two-link invariant to

maintain. **circular list** — `last->next == head`, no `NULL`; traversal stops at `head`; singly and doubly rings for round-robin use, and a linked circular queue over a ring. **ADT reuse** — the same Week 3 stack and Week 5 queue contracts run on chains, with $O(1)$ worst-case ends and unbounded size. **polynomials** — ordered (coeff, exp) nodes; addition is a linear merge by descending exponent; lists win on sparse polynomials, arrays on dense ones. Next week: searching — linear versus binary, and the elementary sorts that put data in the order binary search needs.

Example 7.6.1 (Before-you-go predictions, with the deck's answers). *Answer from the algorithms, then check in the lab (P7). (1) A DLL holds `head` $\leftrightarrow$ 5 $\leftrightarrow$ 15 $\leftrightarrow$ 25; delete the node holding 15 given only a pointer to it: the two links rewritten are 5's `next` = 25 and 25's `prev` = 5, then `free` the 15-node — $O(1)$ worst case. (2) A singly circular list reads `a` $\rightarrow$ `b` $\rightarrow$ `c` $\rightarrow$ `a`: `a->next` = `b` and `c->next` = `a` = `head`; the sentinel that replaces `NULL` is `head` itself — no `NULL`. (3) Add $(2x^2 + 1)$ and $(-2x^2 + 5x)$: the result is $5x + 1$, because the x^2 terms cancel to coefficient 0 and a zero term is never stored, while the unshared $5x$ copies through.*

Exercises

Answer from the algorithms, then check against the Solutions block: pointer surgery is Lab work, but every trace below runs on paper first.

1. DLL insert trace: start from `head` $\leftrightarrow$ 10 $\leftrightarrow$ 30. Insert 20 after the 10-node. Give the four pointer writes in order, the final chain, and the two invariant equalities that must hold.
2. DLL delete trace: on `head` $\leftrightarrow$ 5 $\leftrightarrow$ 15 $\leftrightarrow$ 25, delete the head node (5) and then the tail node (25). For each, write the bypass assignments (including the `head` move) and state the cost.
3. Circular traversal: a singly circular list reads `head` $\rightarrow$ 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ `head`. What does `while (p != NULL)` do, and what is the correct terminating loop that prints 1 2 3 exactly once?
4. Linked queue: from `head` $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\rightarrow$ `NULL` with `tail` at 30, run `dequeue()` twice, then `enqueue(40)`. Give the chain and the location of `head` and `tail` after each operation, and say why the `if (head == NULL) tail = NULL` guard matters. (Genuinely new instance reusing the deck's two-end pattern.)
5. Polynomial addition: add $A = 4x^4 + 2x^2 + 7$ and $B = 3x^3 + 2x^2 + 5$. Give the step-by-step merge, the final chain in descending exponent order, and the worst-case time bound. Which terms, if any, are cancelled?

Solutions

1. **Writes:** (1) `x->next = p->next (30)`; (2) `x->prev = p (10)`; (3) `30->prev = x`; (4) `10->next = x`. **Final chain:** `head` $\leftrightarrow$ 10 $\leftrightarrow$ 20 $\leftrightarrow$ 30. **Invariant:** `20->next->prev = 30->prev = 20` and `20->prev->next = 10->next = 20`. Step 3 is guarded by `p->next != NULL` (here 30 exists); step 4 must come after step 3, which still needs the old successor value.
2. **Delete 5 (head):** `p->prev == NULL`, so `head = p->next (the 15-node)`; then `15->prev = NULL`; `free(p)`. **Delete 25 (tail):** `25->prev` is 15, so `15->next = p->next = NULL`; then `p->next == NULL`, so no backward repair; `free(p)`. $O(1)$ worst case each once `p` is in hand. Both bypasses are $O(1)$; skipping either `free` leaks one box, and touching `p` after is the dangling pointer.
3. **while (p != NULL) loops forever:** no `next` is `NULL` in a ring, so the test never fires. **Correct loop:** `p = head`; `if (p != NULL) do { print p->data; p = p->next; } while (p != head)`; — the `do--while` visits `head` first and stops when the walk returns to it, printing 1 2 3 once. A plain `while (p != head)` would skip node 1 (the head), hence the `do--while` form.

4. **After dequeue 1:** $\text{head} \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$, **tail at 30.** **After dequeue 2:** $\text{head} \rightarrow 30 \rightarrow \text{NULL}$, **tail still 30.** **After enqueue 40:** $\text{head} \rightarrow 30 \rightarrow 40 \rightarrow \text{NULL}$, **tail at 40.** The guard matters when the queue empties: removing the final item sets $\text{head} = \text{NULL}$, and tail must follow to NULL or the next enqueue writes through a stale pointer into freed memory.
5. **Merge: compare 4 vs. 3** $\rightarrow$ **copy** $4x^4$; **2 vs. 3** $\rightarrow$ **copy** $3x^3$; **2 vs. 2** $\rightarrow 2 + 2 = 4$, **emit** $4x^2$; **A has 7, B has 5, both exp 0** $\rightarrow 7 + 5 = 12$, **emit 12.** **Final chain:** $(4,4) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (12,0) \rightarrow \text{NULL}$, i.e. $4x^4 + 3x^3 + 4x^2 + 12$. **No term cancels here (all sums nonzero); worst-case time** $O(n+m)$ The merge visits each source node once and emits at most one output node per step; the equal-exponent step adds coefficients $(2 + 2, 7 + 5)$ and keeps the exponent.

References

- [1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1st edition, 1983.
- [2] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksource, 2nd edition, 2008.
- [3] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.
- [5] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.